52. Evaluating AI Systems in Go

52. Evaluating AI Systems in Go

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `52-eval-harnesses`

Learning objectives

- Build offline eval datasets
- Measure latency/cost/quality
- Automate regression evals in CI
- Separate unit tests from model evals

Prerequisites

Complete prior numbered lessons. Type every example yourself.

What to measure

- **Latency**: p50/p95 end-to-end
- **Cost**: tokens / $ per request
- **Quality**: exact match, rubrics, LLM-as-judge (carefully), retrieval recall@k

Harness

```
for _, tc := range dataset {
    started := time.Now()
    out, err := system.Run(ctx, tc.In)
    // record metrics + score(out, tc.Want)
}
```

Gate releases on eval thresholds. Keep golden sets versioned.

Unit-test pure functions (chunking, cosine); eval the full system less frequently.

Common mistakes (especially from Python instincts)

- Only vibe-checking prompts
- Flaky online evals without seeds
- Optimizing cost while quality collapses

Exercises

Work in `exercises/52-eval-harnesses/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/52-eval-harnesses/` until you have an honest failing attempt.

Self-check

1. Name three metric classes
2. Why golden sets?
3. What belongs in CI vs nightly?

Further reading

- https://go.dev/blog/survey2024-overview